

Sri Lanka Institute of Information Technology



IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 05

Student ID	IT24104150
Student Name	Sathsaranee K.L.U
Campus/ Center name	Malabe
Specialization	CSNE
Batch No	Y2S2.WD.CSNE.01

Activity 1

a)

```
vboxuser@ubuntu:~$ nano lab5.c
vboxuser@ubuntu:~$ gcc -o lab5 lab5.c
vboxuser@ubuntu:~$ ./lab5
Hello from the thread!
Thread has finished execution.
```

b)

- pthread_create()
- pthread_join()
- pthread_exit()
- perror()
- printf()

c)

The program creates a single POSIX thread using `pthread_create()`. The thread executes the function `print_message()`, which simply prints “Hello from the thread!” and then exits using `pthread_exit()`. The main thread waits for the created thread to finish using `pthread_join()` before printing its own completion message.

Activity 2

```
GNU nano 7.2 lab5.c
#include <stdio.h>
#include <pthread.h>

void* print_thread_number(void* arg) {
    long thread_num = (long)arg;
    printf("Hello from thread %ld\n", thread_num);
    return NULL;
}

int main() {
    pthread_t threads[3];

    for (long i = 1; i <= 3; i++) {
        if (pthread_create(&threads[i-1], NULL, print_thread_number, (void*)i) != 0) {
            perror("Failed to create thread");
            return 1;
        }
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("All threads finished.\n");
    return 0;
}
```

```
vboxuser@ubuntu:~$ nano lab5.c
vboxuser@ubuntu:~$ gcc -o lab5 lab5.c
vboxuser@ubuntu:~$ ./lab5
Hello from thread 1
Hello from thread 2
Hello from thread 3
All threads finished.
```

Activity 3

a)

`typedef struct { int a; int b; } Numbers;` creates a new type alias called `Numbers` for the anonymous structure. This allows us to declare variables of type `Numbers` instead of writing `struct {int a; int b;}` every time, making the code cleaner and more readable.

b)

The POSIX thread function (`pthread_start_routine_t`) is defined as `void* (*)(void*)`. That means every thread function must accept a single `void*` parameter and return `void*`.

`void*` is used because it is a generic pointer — it can point to any type of data (`int`, `struct`, `array`, etc.), allowing great flexibility when passing arguments to threads.

c)

This line performs type casting:

- `arg` is of type `void*` (the generic pointer received by the thread function)
- We know that the actual data passed is a pointer to a `Numbers` structure
- `(Numbers*)arg` casts the `void*` back to `Numbers*` so we can access the fields `.a` and `.b`
- Then we store it in a more convenient variable `nums` of type `Numbers*`

Without this cast, the compiler would not allow us to use `->a` or `->b` on a `void*`.

Activity 4

```
GNU nano 7.2 Lab5.c
#include <stdio.h>
#include <pthread.h>

int a, b;
int arr[4];

void* calculateSum(void* arg) {
    int sum = a + b;
    printf("Sum = %d\n", sum);
    return NULL;
}

void* calculateProduct(void* arg) {
    int product = a * b;
    printf("Product = %d\n", product);
    return NULL;
}

void* calculateArraySum(void* arg) {
    int total = 0;
    for(int i = 0; i < 4; i++) {
        total += arr[i];
    }
    printf("Array Total Sum = %d\n", total);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;

    printf("Enter two numbers: ");
```

```
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Enter 4 numbers for the array: ");
    for(int i = 0; i < 4; i++) {
        scanf("%d", &arr[i]);
    }
    pthread_create(&t1, NULL, calculateSum, NULL);
    pthread_join(t1, NULL);

    pthread_create(&t2, NULL, calculateSum, NULL);
    pthread_join(t2, NULL);

    pthread_create(&t3, NULL, calculateSum, NULL);
    pthread_join(t3, NULL);

    return 0;
}
```

```
vboxuser@ubuntu:~$ nano Lab5.c
vboxuser@ubuntu:~$ gcc -oLab5 Lab5.c
vboxuser@ubuntu:~$ ./Lab5
Enter two numbers: 5 4
Enter 4 numbers for the array: 1 2 3 4
Sum = 9
Sum = 9
Sum = 9
```